Tech Leaders · Class 7

Smart Parking Occupancy Dashboard

Sense every bay, show it live, learn from the data

Reliable sensing **ESP32 web server** **Usage analytics**



Did you know?
Big shopping centres use a red or green light above every bay so drivers can see free spaces from far away.

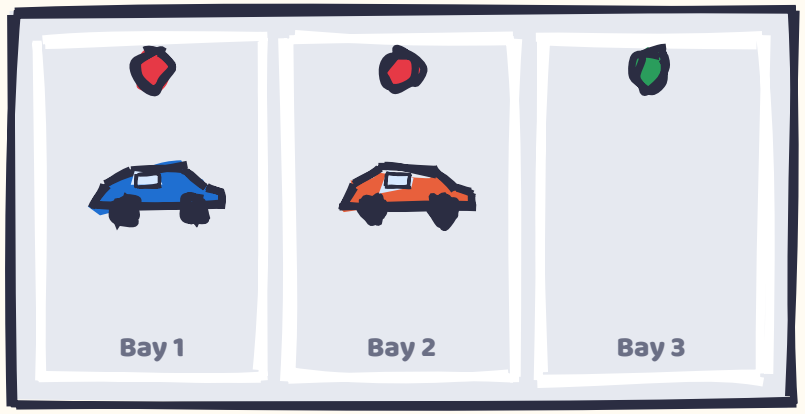
LEARN

The engineering problem

Round and round the car park

Every school morning

Parents circle the car park looking for a space, engines running, while children cross between cars. A sign at the gate saying **'2 spaces free'** or **'FULL'** would save time, fuel and risk. And data could show **how the car park is really used**.



Paint more lines?

A guard with a radio?

Sensors + a live page!

Today: you build a 3-bay model car park whose occupancy appears **live on any phone** on the school Wi-Fi.

Learning objectives

By the end of this class I can...

1	**Reliable sensing:** turn a distance into a **reliable** free / taken decision using a calibrated **threshold** and **debouncing**	**Proof:** a hand waved past a bay does not flip its light
2	**Level shifting:** protect a 3.3 V input from a 5 V signal with a **voltage divider**	**Proof:** calculate the divider output, then measure it
3	**Web server:** serve a web page and a **JSON API** from the ESP32	**Proof:** /status shows valid JSON in a browser
4	**Live dashboard:** update a page **without reloading** using fetch()	**Proof:** the dashboard changes within 2 s of parking a car
5	**Usage data:** calculate **utilisation %** and turn data into a **recommendation**	**Proof:** one data-backed suggestion for our school car park

LEARN

Watch and think

Video time!



Smart Parking Solution (industry demo)

Dahua Technology · about 3 minutes · youtu.be/p_XyDs-k_50

Watch for...

1. How does the system know a space is **free**?
2. How are drivers guided to an empty space?
3. What data could the car park owner use?

Scan to watch

After the video: This is a company's demo video. What would you need to check before believing every claim?

Objective 1 · Reliable sensing

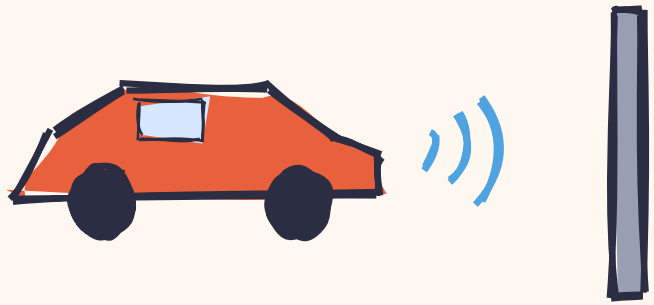
From distance to decision: the threshold

The idea

The sensor gives a **number** (cm). The dashboard needs a **decision** (free or taken). A **threshold** turns one into the other.

Calibrate it: measure an empty bay (e.g. 16 cm) and a parked car (e.g. 4 cm). Put the threshold **halfway**, here about **8 to 10 cm**, so small errors do not flip the answer.

Example: a car's reversing sensor



It does not show you '43 cm'. It turns distance into **decisions**: slow beeps, fast beeps, a solid tone. Each change of sound is a threshold someone calibrated.

Objective 1 · Reliable sensing

Debouncing: don't believe one reading

The idea

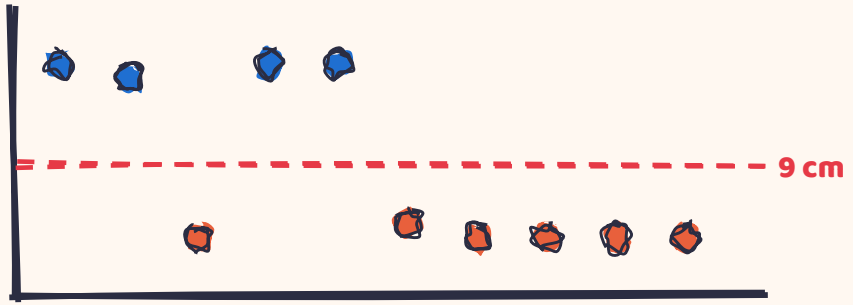
Ultrasonic sensors sometimes give a **wild reading**, and a hand or a bag passing by looks like a car for a moment.

Rule: change the state only when **3 readings in a row** agree. One odd reading is ignored. It costs a fraction of a second of delay and removes most false flips.

Read the graph

Orange dots are under 9 cm. The single dip (3rd dot) is ignored. The run of 3+ at the end flips the bay to taken.

Example: taking attendance



A student who **walks past the classroom door** is not marked present. They must actually **be in their seat** when the teacher looks up. One glimpse is not enough evidence.

Objective 2 · Level shifting

A voltage divider protects the ESP32

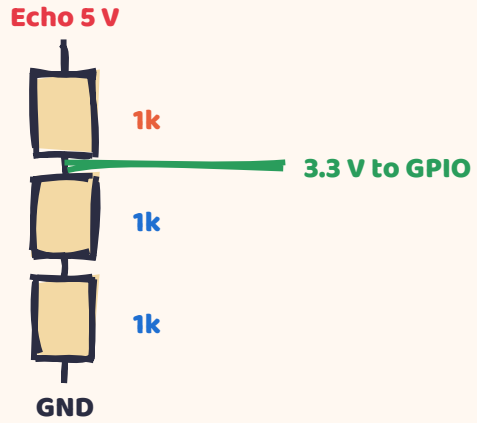
The idea

The HC-SR04 needs 5 V, so its **Echo** pin outputs 5 V. ESP32 pins are rated for **3.3 V**. Two resistors in series share the voltage:

Vout = Vin × R2 ÷ (R1 + R2)

With R1 = 1k and R2 = 2k (two 1k in series):
5 × 2 ÷ 3 = **3.33 V**. Safe.

Example: checking our kit list



Our kit list said **1k + 10k**. Check it: 5 × 10 ÷ 11 = **4.55 V**. That is **too high** for the ESP32. Engineers check the maths, even on official documents. We use 1k + 2k instead.

Objective 3 · Web server

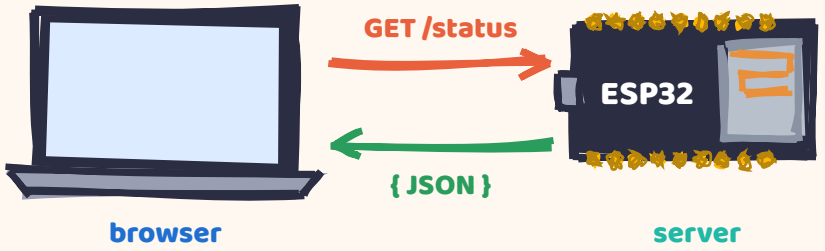
Client and server: request, then response

The idea

The ESP32 runs a tiny **web server**. A browser (the **client**) sends a **request** for a path, and the server sends a **response**.

We make two **routes**:
/ → the dashboard page (HTML)
/status → just the data (JSON)

Example: a restaurant



You (the client) order from the menu: **'one /status please'**. The kitchen (the server) prepares it and the waiter brings back the plate (the response). The menu items are the **routes**. Order something not on the menu: **404**.

LEARN

Objective 3 · Web server

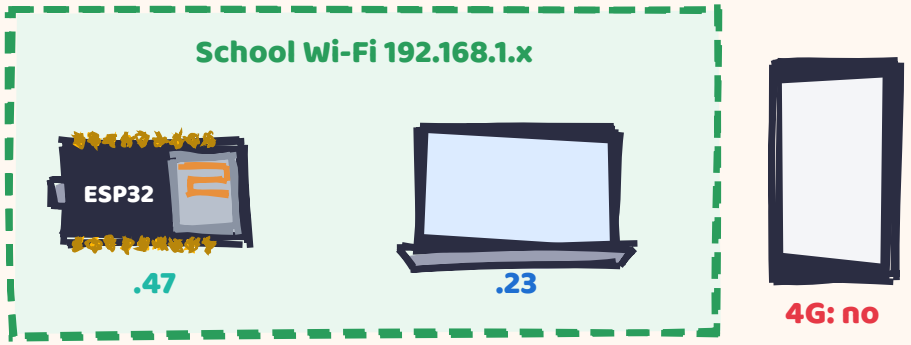
A local IP works only inside the network

The idea

When the ESP32 joins Wi-Fi, the router gives it an **IP address** like **192.168.1.47**. Serial Monitor prints it. Any device **on the same Wi-Fi** can open http://192.168.1.47.

A phone on mobile data is **outside** the network, so it cannot reach it. That is also a safety feature.

Example: rooms inside a school



'Room 12' makes sense to anyone **inside** the school. Someone outside the gate cannot walk to Room 12. A local IP is a room number; the router is the gate.

Objective 3 · Web server

JSON: data with labels

The idea

JSON is text that any program can read. Values have **labels** (keys), and lists use [ ]:

```
{ "free": 1,
  "bays": [
   {"id":1, "occupied":true, "usedMin":12.5},
   ... ] }
```

The browser, a phone app or a Python script can all use the **same** /status.

Example: a cricket scoreboard



245 / 6 means nothing unless you know the labels: **runs** and **wickets**. JSON always sends the labels with the numbers: {"runs":245, "wickets":6}.

LEARN

Objective 4 · Live dashboard

fetch(): update the page without reloading

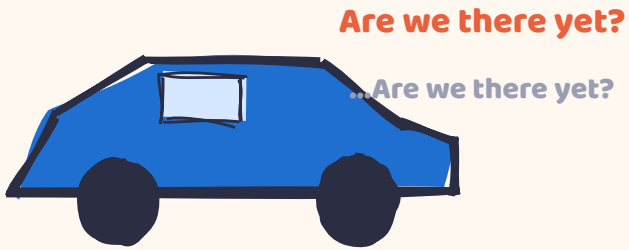
The idea

Reloading the whole page every few seconds flickers and wastes data. Instead, JavaScript asks for **only the data**:

```
setInterval(update, 2000);
```

Every 2 s, **update()** calls **fetch('/status')** and changes just the numbers and colours. This is called **polling**.

Example: "Are we there yet?"



A child in the back seat asks every 2 seconds. It works, but it is tiring for the driver. Polling every 2 s is fine for 3 bays and a few phones; for 10,000 users, servers **push** updates instead (WebSockets).

Objective 5 · Usage data

Utilisation: how busy is each bay?

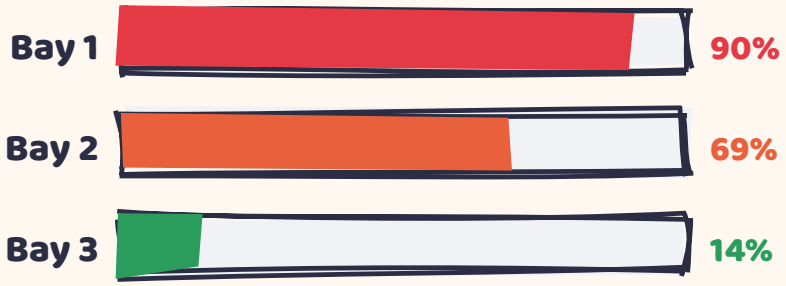
The idea

For every bay we add up the time it was taken.

Utilisation % = time occupied ÷ time observed × 100

A bay at 95% is almost always full. A bay at 10% is nearly wasted space. The **average** hides this, so always look at each bay and at **peak times**.

Example: a worked calculation



Over a school day of 8 hours, Bay 3 was taken for 1 hour 6 minutes = 66 min.
66 ÷ 480 × 100 = **14%**. Bay 1 was taken for 432 min: **90%**.

LEARN

Objective 5 · Usage data

From data to a decision

The idea

Data alone changes nothing. An engineer asks **why** and then **recommends** an action, with evidence:

1. **Observe**: Bay 3 is used 14% of the day.
2. **Explain**: it is furthest from the gate.
3. **Recommend**: make Bay 3 staff-only, keep Bays 1 and 2 for 10-minute drop-off.
4. **Measure again** after the change.

Example: city parking pricing



Observe → Explain → Recommend → Measure

Some cities (for example San Francisco's SFpark pilot) changed parking prices street by street based on sensor data, aiming to keep roughly one space free on every block. The goal was **less circling**, not more money.

LEARN

Objective 5 · Analyse

What does this data say?

Bay	Used min (of 480)	Util %	Peak
Bay 1 (by the gate)	432	90%	07:30 to 08:15
Bay 2 (middle)	331	69%	07:30, 13:30
Bay 3 (far corner)	66	14%	only when 1 and 2 full

Careful

One day of data could be a strange day (exams, rain). How many days would you want before changing the rules?

Your recommendation, in 3 sentences

1. We observed … 2. We think this is because … 3. We recommend … and we will check it worked by measuring …

Privacy plus: ultrasonic sensors count cars without seeing number plates or faces. A camera system would need far more care.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Kit per group
2. Wire 3 sensors with dividers
3. Bay lights and calibration
4. Sense, debounce and light the bay
5. The dashboard page (HTML + JavaScript)
6. Routes, JSON API and the main loop
7. Test the dashboard, then collect data

Build it

Kit per group

ESP32
+ USB cable

3 × HC-SR04
one per bay

9 × 1k resistors
3 dividers (1k + 2k)

3 red + 3 green LEDs
+ 6 × 220 Ω

Cardboard
3-bay car park model

Toy cars
at least 3

Libraries: WiFi and WebServer (both built into the ESP32 core). **Bay size:** about 9 × 14 cm so a toy car fits with the sensor at the back wall.

Kit list fix

The plan said 1k + 10k for the divider. That gives 4.55 V. Use 1k + 2k (two 1k).

MAKE

Objective 2 · Build step 1 of 3

Wire 3 sensors with dividers

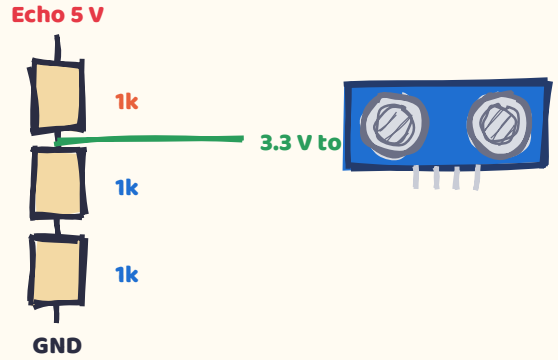
a HC-SR04 VCC to **VIN (5 V)**, GND to GND, on all three.

b TRIG pins (output from ESP32, 3.3 V is enough): Bay 1 **GPIO 5**, Bay 2 **GPIO 19**, Bay 3 **GPIO 22**.

c ECHO → 1k → junction → 1k → 1k → GND. Junction to: Bay 1 **GPIO 18**, Bay 2 **GPIO 21**, Bay 3 **GPIO 23**.

d Before connecting the junction to the ESP32, **measure it** with a multimeter while the sensor runs: about 3.3 V peaks (a meter shows less, since the pulse is short).

e Mount each sensor at the back of its bay, pointing at the entrance.



Check: Divider calculated (3.33 V) and checked before wiring to the ESP32.

Never connect a 5 V Echo straight to an ESP32 pin. It may work for a day, then fail.

MAKE

Objective 1 · Build step 2 of 3

Bay lights and calibration

a Red LEDs (via 220 Ω): Bay 1 **GPIO 25**, Bay 2 **GPIO 32**, Bay 3 **GPIO 16**.

b Green LEDs (via 220 Ω): Bay 1 **GPIO 26**, Bay 2 **GPIO 33**, Bay 3 **GPIO 17**.

c Upload part 1 with Serial prints. Record the **empty** reading and the **car** reading for each bay.

d Set **LIMIT** halfway between them. Write both numbers on the model for the next person.

e Fire one sensor at a time (we update one bay every 100 ms) so echoes do not cross.



Check: Each light changes within about half a second of parking, and a quick hand-wave does not change it.

MAKE

Objective 1 · Code part 1

Sense, debounce and light the bay

```
#include <WiFi.h>
#include <WebServer.h>
const int TRIG[3] = {5, 19, 22},  ECHO[3] = {18, 21, 23};
const int RED_[3] = {25, 32, 16}, GRN_[3] = {26, 33, 17};
const float LIMIT = 9.0;              // cm, from YOUR calibration
bool occ[3];  int agree[3];
unsigned long since[3], usedMs[3];    // state start, total time taken

float readCm(int i) {
  digitalWrite(TRIG[i], LOW);  delayMicroseconds(2);
  digitalWrite(TRIG[i], HIGH); delayMicroseconds(10);
  digitalWrite(TRIG[i], LOW);
  long us = pulseIn(ECHO[i], HIGH, 25000);   // 25 ms timeout
  return us == 0 ? 999 : us / 58.0;          // no echo = far away
}
void updateBay(int i) {
  bool now = readCm(i) < LIMIT;
  if (now != occ[i]) agree[i]++; else agree[i] = 0;
  if (agree[i] >= 3) {                       // 3 in a row: believe it
    if (occ[i]) usedMs[i] += millis() - since[i];  // car left
    occ[i] = now;  since[i] = millis();  agree[i] = 0;
  }
  digitalWrite(RED_[i], occ[i]);
  digitalWrite(GRN_[i], !occ[i]);
}
```

Threshold (O1)

readCm(i) < LIMIT turns a number into a decision. LIMIT comes from your calibration, not from the internet.

Debounce (O1)

agree[i] counts readings that disagree with the current state. Only 3 in a row change it.

Ready for O5

When a car leaves, its parked time is added to **usedMs**.

MAKE

Objective 4 · Code part 2

The dashboard page (HTML + JavaScript)

```
const char PAGE[] = R"html(
<!DOCTYPE html><html><head><meta name="viewport"
 content="width=device-width"></head><body style="font-family:sans-serif">
<h1>School Car Park</h1>
<h2 id="free">loading...</h2>
<div id="bays"></div>
<script>
async function update() {
  const d = await (await fetch('/status')).json();
  document.getElementById('free').textContent =
    d.free === 0 ? 'CAR PARK FULL' : d.free + ' spaces free';
  document.getElementById('bays').innerHTML = d.bays.map(b =>
    `<p style="color:${b.occupied ? 'red' : 'green'}">
     Bay ${b.id}: ${b.occupied ? 'taken' : 'free'} ·
     used ${Math.round(100 * b.usedMin / d.uptimeMin)}%</p>`).join('');
}
setInterval(update, 2000);  update();
</script></body></html>
)html";
```

fetch() (04)

Every 2 s, **update()** asks for /status and rewrites only the numbers and colours. No reload, no flicker.

R"html(…)html"

A C++ **raw string**: the whole web page sits inside the ESP32's memory exactly as written.

Utilisation (05)

usedMin ÷ uptimeMin × 100, calculated in the browser.

MAKE

Objective 3 · Code part 3

Routes, JSON API and the main loop

```
WebServer server(80);
unsigned long usedNow(int i) {          // include a car parked right now
  return usedMs[i] + (occ[i] ? millis() - since[i] : 0);
}
void handleStatus() {                   // the JSON API
  int freeBays = 0;
  String j = "{\"bays\":[";
  for (int i = 0; i < 3; i++) {
    if (!occ[i]) freeBays++;
    j += "{\"id\":" + String(i + 1) + ",\"occupied\":" + (occ[i] ? "true" : "false")
       + ",\"usedMin\":" + String(usedNow(i) / 60000.0, 1) + "}";
    if (i < 2) j += ",";
  }
  j += "],\"free\":" + String(freeBays)
     + ",\"uptimeMin\":" + String(millis() / 60000.0, 1) + "}";
  server.send(200, "application/json", j);
}
void setup() {
  for (int i = 0; i < 3; i++) { pinMode(TRIG[i], OUTPUT); pinMode(ECHO[i], INPUT);
    pinMode(RED_[i], OUTPUT); pinMode(GRN_[i], OUTPUT); }
  Serial.begin(115200);
  WiFi.begin("SchoolLab", "********");
  while (WiFi.status() != WL_CONNECTED) delay(300);
  Serial.println(WiFi.localIP());       // open this address in a browser
  server.on("/", []() { server.send(200, "text/html", PAGE); });
  server.on("/status", handleStatus);
  server.begin();
}
void loop() {
  server.handleClient();
  static unsigned long t = 0;  static int i = 0;
  if (millis() - t > 100) { t = millis(); updateBay(i); i = (i + 1) % 3; }
}
```

Two routes (O3)

/ sends the page, **/status** sends JSON. Test /status first: if it is wrong, the page will be too.

Never block

One bay every 100 ms and **handleClient()** every loop, so the page always answers fast.

Objectives 4 + 5 · Build step 3 of 3

Test the dashboard, then collect data

Test

1. Open **http://<IP>/status**. Is it valid JSON? (Paste it into jsonlint.com.)
2. Open **http://<IP>/** on two phones at once.
3. Park a car: both phones change within 2 s?
4. Fill all 3 bays: **CAR PARK FULL**?

Collect (simulated school day)

Run for **20 minutes**. Each minute = 1 school hour. A ‘traffic controller’ reads a script of arrivals and departures. Screenshot the dashboard at the end and record the % for each bay.

Why simulate?
We cannot wait a whole day. Scaling time is what engineers do with models: wind tunnels, crash tests, this.

Bay	Used min	Util %
Bay 1		
Bay 2		
Bay 3		

Test like an engineer

Troubleshooting

Symptom	Likely cause
Bay always 'taken'	LIMIT too high or sensor sees the side wall: aim it, recalibrate
Readings jump between bays	sensors fired together: keep one bay every 100 ms
Page will not open	phone on mobile data or a different Wi-Fi
Page loads, never updates	JavaScript error: open the browser console (F12)
Echo reads 0 always	divider wired wrong; measure the junction voltage

Debug checklist

1. /status valid JSON?
2. Serial print readCm()
3. Same Wi-Fi?
4. Console errors?

Debug in layers: **sensor** → **state** → **JSON** → **page**. Find the first layer that is wrong; everything after it will be wrong too.

MAKE

Level up!

Apply what you learned

Choose your challenge

- **Gate sign:** a second ESP32 with an OLED at the 'gate' that reads /status and shows FREE: 2 (O3, O4).
- **Average stay:** count arrivals and show the average parking time per bay (O5).
- **Smarter sensing:** use the median of 5 readings instead of 3-in-a-row. Compare (O1).
- **Level-shifter chip:** replace the dividers with a proper level-shifter module and explain why it is better (O2).

Think like an engineer

A school has 120 spaces. Would you use 120 ultrasonic sensors? What else could count cars at the entrance and exit?

Boss level

Log every arrival and departure with a timestamp to a CSV you can download from /log.csv.

Objective check

Can I really do it? Prove it.

01	An empty bay reads 18 cm and a car reads 5 cm. Where do you set LIMIT, and why do we need 3 readings in a row?	○ ○ ○
02	What voltage does a 2.2k + 3.3k divider give from 5 V? Is it safe for the ESP32?	○ ○ ○
03	What is the difference between the **/** route and the **/status** route? Why can't a phone on 4G open it?	○ ○ ○
04	Explain what fetch() does every 2 seconds, and why it is better than reloading the page.	○ ○ ○
05	A bay was taken for 120 of 400 minutes. What is its utilisation? Give one recommendation based on it.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: O1: about 11 to 12 cm. O2: 5 × 3.3 ÷ 5.5 = **3.0 V**, safe. O5: **30%**.

Show what you know

What did we learn?

- [x] A calibrated **threshold** + **3-in-a-row debounce** = a reliable decision
- [x] Vout = Vin × R2 ÷ (R1 + R2): **1k + 2k** makes 5 V into 3.33 V
- [x] The ESP32 serves a page at **/** and JSON at **/status**
- [x] **fetch()** every 2 s updates the page without reloading
- [x] **Utilisation %** per bay turns data into a recommendation

Exit ticket: Why did we not trust the 1k + 10k divider from the kit list?

At home

Watch a real car park for 10 minutes. Which spaces fill first, and why?



Next class: TL-08 Two-Factor Smart Locker